State Space Search

Recursive enumeration of expressions

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 14 Tuesday, October 20, 2026

Today

Plan

1. A puzzle to warm up: **Make 24**.
2. **States, moves, and goals**: how to describe a search problem.
3. How big is the search? Counting the **branches**.
4. Recursive enumeration: **choose, explore, undo**.
5. A small recursive search you can read in one slide.
6. A warning about **floating-point** numbers.

- Exercise 6 is due today. Exercise 7 (Permutation Trace) is posted on Thursday.
- Today leads into Project 3, which is posted **after** the midterm. State-space search is **not** on the midterm (Oct 27).

Warm-up: Make 24

Use each of the numbers **exactly once**, with $+$, $-$, $\times$, $\div$ and parentheses, to make **24**.

(a) $\{1, 2, 3, 4\}$ **(b)** $\{3, 3, 8, 8\}$ **(c)** $\{5, 5, 5, 1\}$

Try them with a neighbor for two minutes. Notice *how* you search: do you try combinations at random, or follow a pattern?

Warm-up: Make 24

Use each of the numbers **exactly once**, with $+$, $-$, $\times$, $\div$ and parentheses, to make **24**.

(a) $\{1, 2, 3, 4\}$ **(b)** $\{3, 3, 8, 8\}$ **(c)** $\{5, 5, 5, 1\}$

Try them with a neighbor for two minutes. Notice *how* you search: do you try combinations at random, or follow a pattern?

Answers.
(a) $1 \times 2 \times 3 \times 4 = 24$. Also $(1 + 3) \times (2 + 4)$.
(b) $8 \div (3 - 8 \div 3) = 24$. Few people find this one by hand.
(c) $5 \times (5 - 1 \div 5) = 24$.

A computer does not need a trick. It tries **every** way to combine the numbers, and checks each result. Today we learn how to organize that search.

Search Problems: States, Moves, Goal

Analogy. Think of a board game. At any moment the board is a **state**. A legal move takes you to a new state. You win when the board matches a **goal**.

	Maze (P2)	**Make 24**
State	the cell you stand on	the numbers (and expressions) left on the table
Move	step to a neighbor	pick two items, combine them with one operator
Goal	the cell E	one item left, equal to 24
Dead end	no unvisited neighbor	one item left, not 24

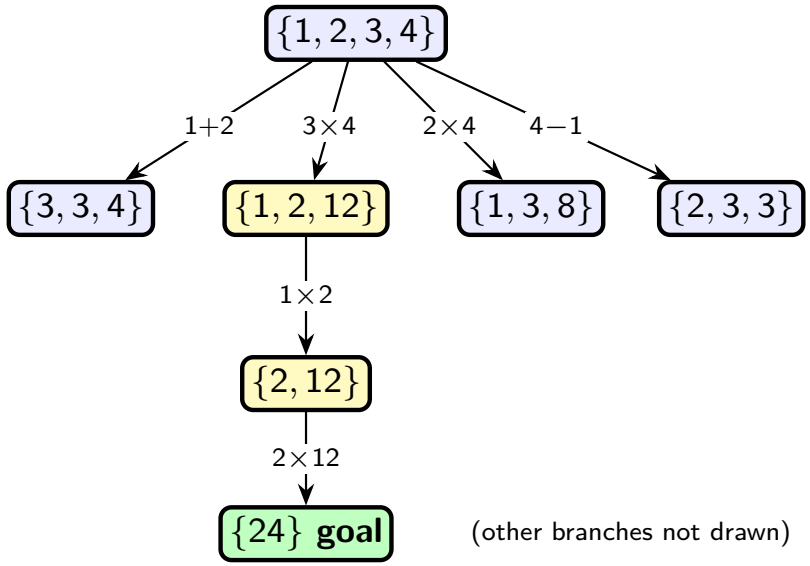
The set of all states, joined by moves, is the **state space**. A **state-space search** walks through it looking for a goal. DFS and BFS from Classes 11–12 were state-space searches on a maze: the new part today is that the states are not cells.

Make 24 as a State Space

Each item remembers its **value** and its **text**, so that we can print the answer.

Start: four items $1, 2, 3, 4$.
Move: remove two items a, b; add one new item: $a + b$, $a - b$, $b - a$, $a \times b$, $a \div b$, or $b \div a$ (skip a divide by 0).
After 3 moves: one item is left; it is the goal if it equals 24.

Each move makes the table **one item smaller**: so the search always ends.



Question: How Many Moves From the Start?

Start with four items. A move picks **two** of them and combines them with one result.

(a) How many *pairs* can you pick from 4 items?
(b) For one pair $\{a, b\}$, how many operation choices are there?
(c) So how many possible first moves are there, at most?

Question: How Many Moves From the Start?

Start with four items. A move picks **two** of them and combines them with one result.

(a) How many *pairs* can you pick from 4 items?
(b) For one pair $\{a, b\}$, how many operation choices are there?
(c) So how many possible first moves are there, at most?

(a) 6 pairs: $\binom{4}{2} = 6$. (Pick 2 from a, b, c, d: ab, ac, ad, bc, bd, cd.)

(b) 6 operation choices: $a + b$, $a \times b$, $a - b$, $b - a$, $a \div b$, $b \div a$ (if neither division is by 0). Addition and multiplication are **symmetric**, so one choice each; subtraction and division give two.

(c) $6 \times 6 = \mathbf{36}$ first moves, at most.

Question: How Big Is the Whole Search?

After the first move, the table has **3** items. After the second, **2**.

(a) How many moves are possible with 3 items on the table? With 2 items?

(b) How many *complete sequences* of 3 moves are there, in total?

Items on table	Pairs	Choices per pair	Moves
4			
3			
2			

Question: How Big Is the Whole Search?

After the first move, the table has **3** items. After the second, **2**.

(a) How many moves are possible with 3 items on the table? With 2 items?
(b) How many *complete sequences* of 3 moves are there, in total?

Items on table	Pairs	Choices per pair	Moves
4	6	6	36
3	3	6	18
2	1	6	6

Total sequences: at most $36 \times 18 \times 6 = \mathbf{3888}$ (fewer if a division by 0 is skipped). A computer checks them in microseconds, so brute force is fine for four numbers. **Rule:** multiply the number of choices at each level.

Recursive Enumeration: Choose, Explore, Undo

Every recursive search in this unit has the same three-part rhythm.

```
bool search(State& s) {
    if (isGoal(s))   { return true; }     // base case: success
    if (isDeadEnd(s)) { return false; }  // base case: failure
    for (each legal move m) {
        apply(s, m);                      // 1. choose
        if (search(s)) { return true; }   // 2. explore
        undo(s, m);                       // 3. undo
    }
    return false;                         // every move failed
}
```

- **Analogy:** exploring a cave with a ball of string. You walk down a tunnel; if it is a dead end, you wind the string back and try the next tunnel.
- **Undo** puts the state back exactly as it was, so the next move starts from the same place. If the state is passed *by value*, the copy is discarded automatically.

A Small Search: Left to Right

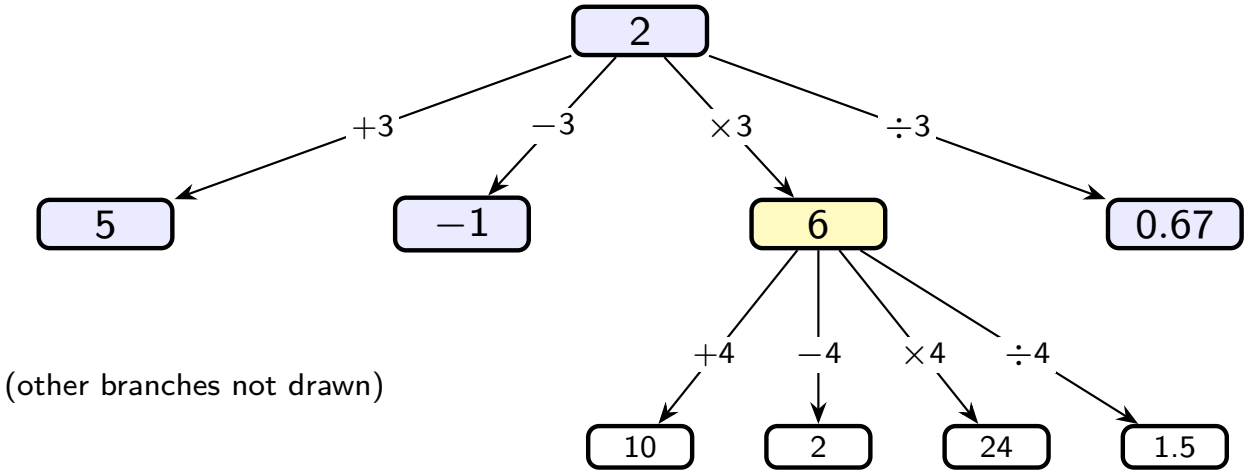
Toy problem. Numbers stay in a **fixed order**. Put $+, -, \times, \div$ between consecutive numbers and evaluate **strictly from left to right**, ignoring the usual operator precedence. Can we reach the target?

```
bool search(const std::vector<int>& v, int k, double value, double
     target) {
    if (k == static_cast<int>(v.size())) {      // all numbers used
        return std::abs(value - target) < 1e-9;
    }
    double x = v[k];
    return search(v, k + 1, value + x, target)
        || search(v, k + 1, value - x, target)
        || search(v, k + 1, value * x, target)
        || (x != 0 && search(v, k + 1, value / x, target));
}
// first call: search(v, 1, v[0], 24)
```

State = *(k, value)*: `k` numbers used, `value` so far. `||` stops at the first `true` (early exit). `value` is passed **by value**, so there is nothing to undo.

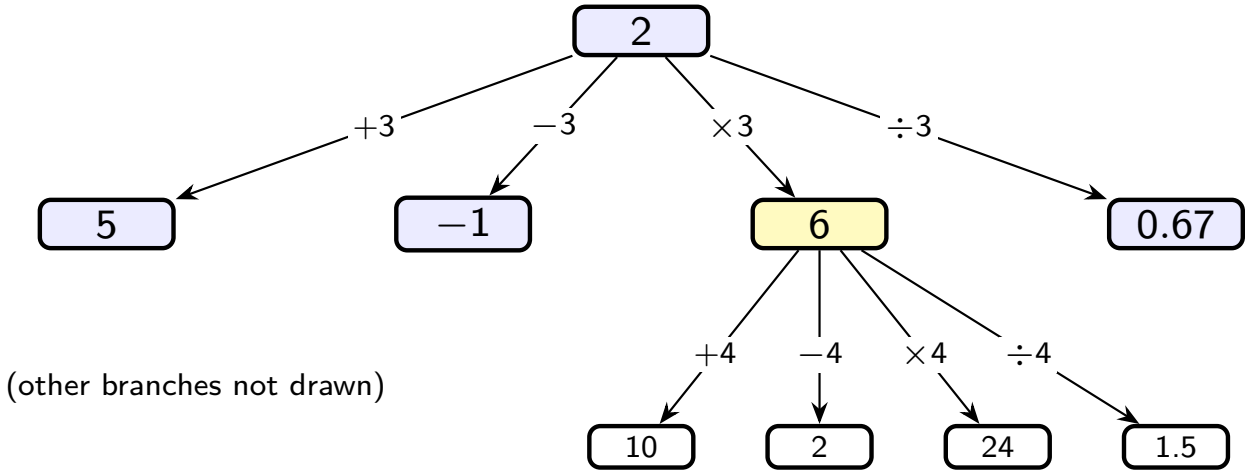
Question: What Does `search` Return?

`v = {2, 3, 4}`, the first call is `search(v, 1, 2, 24)`. How many leaves (complete expressions) does the search have in the worst case? Is `true` returned?



Question: What Does `search` Return?

`v = {2, 3, 4}`, the first call is `search(v, 1, 2, 24)`. How many leaves (complete expressions) does the search have in the worst case? Is `true` returned?



The picture shows the first level and the leaves under one branch. The branch $2 \times 3 = 6$ has leaves $10, 2, \mathbf{24}, 1.5$ (from $+4, -4, \times 4, \div 4$). The total is $4 \times 4 = \mathbf{16}$ leaves. Yes: $2 \times 3 \times 4 = 24$, so `true` is returned.

Two Searches: Left to Right vs. Free Order

Both are state-space searches. They differ in what a **move** is.

	Left to right (toy)	**Free order** (Make 24)
Move	next number, one of $+, -, \times, \div$	any two items, one of 6 operation choices
Numbers	fixed order	any order
Grouping	left-associated: strictly left to right	any, e.g. $8 \div (3 - 8 \div 3)$
Leaves, $n = 3$ / $n = 4$	16 / 64	at most 108 / 3888

Question. Can the left-to-right search make 24 from $\{3, 3, 8, 8\}$, with the numbers in any order?

Two Searches: Left to Right vs. Free Order

Both are state-space searches. They differ in what a **move** is.

	Left to right (toy)	**Free order** (Make 24)
Move	next number, one of $+, -, \times, \div$	any two items, one of 6 operation choices
Numbers	fixed order	any order
Grouping	left-associated: strictly left to right	any, e.g. $8 \div (3 - 8 \div 3)$
Leaves, $n = 3$ / $n = 4$	16 / 64	at most 108 / 3888

Question. Can the left-to-right search make 24 from $\{3, 3, 8, 8\}$, with the numbers in any order?
No. A solution is $8 \div (3 - 8 \div 3) = 24$, which needs a grouping that left-to-right evaluation cannot build. Free order can build it. Your Project 3 search is the free-order one.

Growth: Why Search Is Expensive

In the toy search each new number has **4** choices $(+, -, \times, \div)$, so n numbers give 4^{n-1} leaves.

n	leaves 4^{n-1}	**Your guess?**
3	16	
4		
5		
6		
8		
12		

Growth: Why Search Is Expensive

In the toy search each new number has **4** choices $(+, -, \times, \div)$, so n numbers give 4^{n-1} leaves.

n	leaves 4^{n-1}	**Your guess?**
3	16	
4	64	
5	256	
6	1 024	
8	16 384	
12	4 194 304	

Each extra number multiplies the work by 4: **exponential growth**. Free-order Make 24 grows even faster (at most 3888 for four numbers; about 21 million for six). On Oct 29 we will **prune**: stop a branch early when it cannot succeed.

A Trap: Floating-Point Numbers

Question. What do these lines print?

```
double a = 0.1 + 0.2;
std::cout << (a == 0.3);                // (A) 1   (B) 0
double b = 8.0 / (3.0 - 8.0 / 3.0);
std::cout << (b == 24.0);               // (C) 1   (D) 0
```

A Trap: Floating-Point Numbers

Question. What do these lines print?

```
double a = 0.1 + 0.2;
std::cout << (a == 0.3);              // (A) 1   (B) 0
double b = 8.0 / (3.0 - 8.0 / 3.0);
std::cout << (b == 24.0);             // (C) 1   (D) 0
```

Answers: (B) and (D), both print 0. Computers store most decimals in binary, which cannot represent 0.1 or 8/3 exactly. Here a is 0.30000000000000004 and b is 23.999999999999.

Analogy: writing 1/3 as 0.333 on paper. Two different roads to "one third" may land on slightly different decimals.

Fix: compare with a small **tolerance**:

```
bool nearlyEqual(double x, double y) { return std::abs(x - y) < 1e-9;
     }
```

Avoid == when testing whether a computed `double` equals a target.

Use a tolerance. This matters for the Make 24 goal test:

$8 \div (3 - 8 \div 3)$ must count as 24.

Practice: Count the Search

1. A toy search has $n = 5$ numbers in a fixed order and 4 operators. How many leaves?
2. Free-order Make 24 with **3** numbers: how many sequences of moves are there?
3. In the toy search, what does `x != 0 && ...` protect against?
4. Why is `value == target` a bad goal test for `double`?
5. In the recursive template, what would go wrong if we **forgot** the *undo* step in a version that changes one shared state?

Practice: Count the Search

1. A toy search has $n = 5$ numbers in a fixed order and 4 operators. How many leaves?
2. Free-order Make 24 with **3** numbers: how many sequences of moves are there?
3. In the toy search, what does `x != 0 && ...` protect against?
4. Why is `value == target` a bad goal test for `double`?
5. In the recursive template, what would go wrong if we **forgot** the *undo* step in a version that changes one shared state?

1. $4^4 = 256$.
2. 3 items: 3 pairs $\times$ 6 $=$ 18; then 2 items: $1 \times 6 = 6$. At most $18 \times 6 = \mathbf{108}$.
3. Dividing by 0: the guard skips that branch. (A `double` divided by 0 gives `inf` or `nan`, and later steps would be meaningless.)
4. Rounding errors: a result that is mathematically 24 may be 23.99999999999999.
5. The next move would start from a leftover state, not the state we branched from: we would explore wrong or missing branches.

Common Mistakes

- Forgetting a base case, or testing the goal *after* using the state in a way that is invalid.
- Missing the **undo** when state is shared (passed by reference). Passing by value is safer but copies more.
- Comparing `double` with ==. Use a tolerance.
- Dividing by zero, or by a value that is *nearly* zero.
- Counting incorrectly: $a - b$ and $b - a$ are *separate* operation choices; $a + b$ and $b + a$ are the **same** choice.
- Returning `false` too early: only return `false` after **all** moves have been tried.

Summary and Preview

- A search problem = **states**, **moves**, and a **goal test**.
- A recursive search tries each move: **choose, explore, undo**.
- Counting levels: multiply the choices (at most $36 \times 18 \times 6 = 3888$; 4^{n-1}).
- Use a tolerance for `double`.

Coming up

Thursday, Oct 22: permutations and operator combinations (Exercise 7).
Oct 27: Midterm (through DFS/BFS).
Oct 29: pruning and early termination, then Project 3.